

Guía Fundamental de Formularios en PHP

1. Conceptos Básicos

¿Qué es un formulario?

Un formulario HTML permite al usuario enviar datos al servidor. PHP procesa estos datos en el backend.

Métodos de envío

- **GET:** Los datos se envían en la URL (visibles). Usado para búsquedas y filtros.
- **POST:** Los datos se envían en el cuerpo de la petición (no visibles en URL). Usado para datos sensibles.

2. Estructura Básica de un Formulario HTML

```
<form action="procesar.php" method="POST">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <button type="submit">Enviar</button>
</form>
```

Atributos importantes:

- `action` : URL del archivo PHP que procesará los datos
- `method` : GET o POST
- `name` : Clave para acceder al valor en PHP
- `enctype` : Necesario para subir archivos (`multipart/form-data`)

3. Captura de Datos en PHP

Superglobales

PHP usa arrays superglobales para acceder a los datos:

```
// Datos enviados por POST
$_POST['nombre_campo']

// Datos enviados por GET
$_GET['nombre_campo']

// Ambos métodos
$_REQUEST['nombre_campo'] // No recomendado (menos seguro)

// Archivos subidos
$_FILES['nombre_campo']
```

Ejemplo de captura

```
<?php
// procesar.php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nombre = $_POST['nombre'];
    $email = $_POST['email'];

    echo "Hola, $nombre. Tu email es: $email";
}
?>
```

4. Validación de Datos

Validación del lado del servidor (OBLIGATORIA)

```
<?php
// Verificar que los campos existan
if (isset($_POST['nombre']) && isset($_POST['email'])) {
    $nombre = $_POST['nombre'];
    $email = $_POST['email'];

    // Validar que no estén vacíos
    if (empty($nombre)) {
        $errores[] = "El nombre es obligatorio";
    }

    // Validar formato de email
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errores[] = "Email no válido";
    }

    // Validar longitud
    if (strlen($nombre) < 3) {
        $errores[] = "El nombre debe tener al menos 3 caracteres";
    }

    if (empty($errores)) {
        // Procesar datos
        echo "Datos válidos";
    } else {
        // Mostrar errores
        foreach ($errores as $error) {
            echo "<p>$error</p>";
        }
    }
}
?>
```

Filtros comunes

```
// Validar entero
filter_var($edad, FILTER_VALIDATE_INT);

// Validar URL
filter_var($url, FILTER_VALIDATE_URL);

// Validar booleano
filter_var($acepta, FILTER_VALIDATE_BOOLEAN);

// Sanitizar string (eliminar etiquetas HTML)
filter_var($texto, FILTER_SANITIZE_STRING);

// Sanitizar email
filter_var($email, FILTER_SANITIZE_EMAIL);
```

5. Tipos de Campos Comunes

Inputs de texto

```
$nombre = trim($_POST['nombre']); // trim() elimina espacios
$apellido = trim($_POST['apellido']);
```

Checkbox

```
// HTML
<input type="checkbox" name="acepta" value="1">

// PHP
$acepta = isset($_POST['acepta']) ? true : false;
```

Radio buttons

```
// HTML
<input type="radio" name="genero" value="M"> Masculino
<input type="radio" name="genero" value="F"> Femenino

// PHP
$genero = $_POST['genero']; // "M" o "F"
```

Select

```
// HTML
<select name="pais">
  <option value="es">España</option>
```

```

        <option value="mx">México</option>
    </select>

    // PHP
    $pais = $_POST['pais'];

```

Múltiples valores (checkboxes o select múltiple)

```

// HTML
<input type="checkbox" name="intereses[]" value="deportes">
<input type="checkbox" name="intereses[]" value="musica">

// PHP
$intereses = isset($_POST['intereses']) ? $_POST['intereses'] : [];
foreach ($intereses as $interes) {
    echo $interes . "<br>";
}

```

6. Subida de Archivos

HTML

```

<form action="upload.php" method="POST" enctype="multipart/form-data">
    <input type="file" name="archivo">
    <button type="submit">Subir</button>
</form>

```

PHP

```

<?php
if (isset($_FILES['archivo'])) {
    $archivo = $_FILES['archivo'];

    // Información del archivo
    $nombre = $archivo['name'];
    $tipo = $archivo['type'];
    $tamaño = $archivo['size'];
    $tmp = $archivo['tmp_name'];
    $error = $archivo['error'];

    // Validar que no haya errores
    if ($error === UPLOAD_ERR_OK) {
        // Validar tamaño (ej: 2MB máximo)
        if ($tamaño <= 2097152) {
            // Validar tipo
            $extension = pathinfo($nombre, PATHINFO_EXTENSION);
            $permitidas = ['jpg', 'jpeg', 'png', 'pdf'];

```

```

        if (in_array(strtolower($extension), $permitidas)) {
            // Generar nombre único
            $nuevo_nombre = uniqid() . '.' . $extension;
            $destino = 'uploads/' . $nuevo_nombre;

            // Mover archivo
            if (move_uploaded_file($tmp, $destino)) {
                echo "Archivo subido correctamente";
            } else {
                echo "Error al mover el archivo";
            }
        } else {
            echo "Tipo de archivo no permitido";
        }
    } else {
        echo "Archivo demasiado grande";
    }
} else {
    echo "Error en la subida: " . $error;
}
}
?>

```

7. Mantener Valores en el Formulario

Después de un error, mostrar los valores que el usuario ya había ingresado:

```

<form method="POST">
    <input type="text" name="nombre"
        value="<?php echo isset($_POST['nombre']) ?
htmlspecialchars($_POST['nombre']) : ''; ?>">

    <input type="email" name="email"
        value="<?php echo isset($_POST['email']) ?
htmlspecialchars($_POST['email']) : ''; ?>">

    <button type="submit">Enviar</button>
</form>

```

8. Redirección después de Procesar

```

<?php
// Después de procesar correctamente
header('Location: exito.php');
exit();
?>

```

9. Ejemplo Completo

```

<?php
session_start();
$errores = [];

// Generar token CSRF
if (empty($_SESSION['token'])) {
    $_SESSION['token'] = bin2hex(random_bytes(32));
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Verificar token CSRF
    if (!hash_equals($_SESSION['token'], $_POST['token'] ?? '')) {
        die("Error de seguridad");
    }

    // Capturar y validar
    $nombre = trim($_POST['nombre'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $edad = $_POST['edad'] ?? '';

    if (empty($nombre)) {
        $errores[] = "El nombre es obligatorio";
    }

    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errores[] = "Email inválido";
    }

    if (!filter_var($edad, FILTER_VALIDATE_INT, ['options' => ['min_range' => 18]]))
{
        $errores[] = "Debes ser mayor de 18 años";
    }

    // Si no hay errores, procesar
    if (empty($errores)) {
        // Guardar en base de datos, enviar email, etc.
        $_SESSION['mensaje'] = "Registro exitoso";
        header('Location: exito.php');
        exit();
    }
}
?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Formulario</title>
</head>
<body>
    <?php if (!empty($errores)): ?>

```

```

        <div style="color: red;">
            <?php foreach ($errores as $error): ?>
                <p><?php echo htmlspecialchars($error); ?></p>
            <?php endforeach; ?>
        </div>
    <?php endif; ?>

    <form method="POST">
        <input type="hidden" name="token" value="<?php echo $_SESSION['token']; ?>">

        <label>Nombre:</label>
        <input type="text" name="nombre"
            value="<?php echo htmlspecialchars($nombre ?? ''); ?>">
        <br>

        <label>Email:</label>
        <input type="email" name="email"
            value="<?php echo htmlspecialchars($email ?? ''); ?>">
        <br>

        <label>Edad:</label>
        <input type="number" name="edad"
            value="<?php echo htmlspecialchars($edad ?? ''); ?>">
        <br>

        <button type="submit">Enviar</button>
    </form>
</body>
</html>

```

10. Buenas Prácticas

1. **Siempre valida en el servidor:** La validación del cliente (JavaScript) puede ser evadida
2. **Usa HTTPS:** Especialmente para datos sensibles
3. **Nunca confíes en los datos del usuario:** Siempre valida y sanitiza
4. **Usa consultas preparadas:** Para prevenir inyección SQL
5. **Implementa límites de tasa:** Para prevenir spam
6. **Registra intentos sospechosos:** Para detectar ataques
7. **Usa tokens CSRF:** Para formularios que modifican datos
8. **Valida tipos de archivo:** No solo la extensión, también el MIME type
9. **Establece permisos correctos:** Para carpetas de subida (755 o 750)
10. **Mantén PHP actualizado:** Para tener las últimas correcciones de seguridad

11. Recursos Adicionales

- Documentación oficial: [php.net](https://www.php.net)